# A Workbook Introduction to basic R

Michael Hills (May 26, 2009)

Revised by Aurelio Tobias for the EEPE Residential Summer Course

(June 15, 2026)

# Preface

This workbook was originally developed by Michael Hills (1937–2021), a central figure in shaping the way statistics has been taught in the European Educational Programme in Epidemiology Residential Summer Course, where he taught from its inception in 1991. Michael was renowned for his ability to explain complex ideas in a clear and simple way, and he strongly encouraged students to learn through direct interaction with data and software. In this spirit, the workbook provides a concise introduction to R for readers who may be encountering the software for the first time.

We have reviewed the workbook mainly for EEPE students who are new to R. Chapters 9, 10 and 11 cover most of the topics addressed during the first two weeks of the course. Following Michael's philosophy that statistical software is best learned by using it, we assume that the reader is seated in front of a computer running R. For this reason, we have not felt it necessary to print the output that follows each command.

The workbook mostly uses base R commands, although—given that one of R's strengths is the large number of user-written packages—we have also incorporated a few of these when appropriate. The datasets and additional files are an integral part of this workbook. They can be downloaded with the EEPE course materials, also available from the GitHub repository at: `https://github.com/aureliotobias/Rworkbook`.

Aurelio Tobias
E-mail: `aurelio.tobias@idaea.csic.es`

Neil Pearce
E-mail: `neil.pearce@lshtm.ac.uk`

# Index of Chapters

**0 Getting started** **4**

**1 Basic tools** **5**

**2 Data frames** **10**

**3 Reading data from files** **12**

**4 Factors** **14**

**5 Tables** **15**

**6 Graphics in R** **17**

**7 High level graphics functions** **18**

**8 Low level graphics functions** **21**

**9 Effects** **23**

**10 Linear models** **25**

**11 Survival curves** **28**

**12 Writing simple functions** **31**

**13 More about data manipulation** **32**

**14 Packages** **33**

**15 References** **34**

# 0 Getting started

- R is driven by typing instructions.
- R is case sensitive.
- You can use the ↑ and ↓ keys to recall and edit previous instructions.

R allows you to build powerful procedures from simple building blocks. The building blocks are objects and functions.

- All data in R is represented by objects.
- You the user call functions.
- Functions operate on objects to create new objects.
- Objects are grouped into classes.

All the obvious mathematical functions are available in R:

$$\log(x),\ \exp(x),\ x+y,\ \frac{x}{y},\ x^y \text{ or } x^{**y},\ \min(),\ \sum(),\ \text{mean}(),\ \text{round}(),\ldots$$

Solutions to the exercises are in the directory `"solutions"`, in the form of R scripts such as `btools.r`, etc.

# 1 Basic tools

## Summary

`getwd()` Returns the working directory.
`setwd()` Sets the working directory.
`help(f)` Opens the help page for the function `f()`.
`?f` Does the same.
`help.search()` Searches for documentation matching a given string.
`c()` Combines the arguments into a vector.
`seq()` Creates a vector composed of a regular sequence of numbers.
`1:5` The sequence `1 2 3 4 5`.
`rep(x, n)` Repeats a vector `x`, `n` times.
`length(x)` Returns the length of `x`.
`x[i]` The `i`'th element of `x`. Note the square brackets.
`objects()` Shows the contents of your workspace.
`ls()` Does the same as `objects()`.
`rm(x)` Removes `x` from your workspace.
`rm(list=ls())` Removes all objects from your workspace.
`matrix(x)` Converts a vector `x` to a matrix.
`cbind()` Combines vectors as the columns of a matrix.
`rbind()` Combines vectors as the rows of a matrix.
`list()` Creates a list.
`str()` Displays the internal structure of an object.
`unique(x)` Removes duplicate elements from `x`.
`sort(x)` Sorts elements of the vector `x` in order.
`TRUE`/`FALSE` Values returned by logical expressions.
`which()` Returns the `TRUE` indices of a logical object.
`runif(20)` Returns 20 numbers between 0 and 1 from a uniform distribution.
`rnorm(20)` Returns 20 numbers from a normal distribution with mean 0 and sd 1.
`as.Date()` Converts dates as strings to standard form.
`format("date")` Formats a date in different ways.
`help(strftime)` Help for different date formats.
`cal.yr()` Converts dates as strings to decimal years (from `Epi` library).
`q()` Quits R — answer `n` to the question "save workspace image".

## Examples

### 1. The working directory.

```
> getwd()
```

### 2. Simple arithmetic.

```
> 2 + 2
> x <- 2 + 2
```

Any object already called `x` will be overwritten, without warning.

### 3. Vectors.

A vector is an ordered collection of elements which are either all numbers or all strings.

```
> x
```

```
> y <- c(6, 2, 5)
> y
> length(y)
> c(x, y)
> x + y
```

The shorter vector is cycled until it has the same length as the longer.

```
> 1:5
> rep(3, 20)
> seq(from = 20, to = 50, by = 10)
```

The three named arguments can be given in any order. Their values are set using `=`.

```
> seq(20, 50, 10)
```

The order of the arguments matters when names are omitted.

**4. Logical expressions.**

```
> 7 > 5
> (7 > 5) & (2 > 3)
> (7 > 5) | (2 > 3)
> y
> y > 2
> which(y > 2)
> !(y > 2)
> y == 2
> !(y == 2)
```

Logical equals is written as `==` and not equal is written as `!=`.

**5. Indexing a vector.**

```
> y[1]
> y[2]
> y[1:2]
> y[c(1, 3)]
> y[-1]
> y[-(1:2)]
> y[-c(1, 3)]
> y[y > 2]
> y[c(1, 1, 1, 3, 3)]
```

Guess the result of the next one before you do it.

```
> c("A", "B")[c(1, 1, 1, 2, 2)]
```

`c("A","B")` is a vector indexed by `c(1,1,1,2,2)`.

**6. Lists.**

```
> la <- list(212, "George")
> la
> str(la)
```

The elements of a list can be both numeric and character, and even other lists. The function `str()` is the best way of looking at the contents of a list.

**7. Indexing a list.**

```
> la[1]
> la[2]
> la[1:2]
> la[[1]]
> la[[2]]
```

**8. Named lists.**

```
> lb <- list(first = 212, second = "George")
> lb
> str(lb)
> lb[[1]]
> lb$first
```

**9. Matrices.**

```
> matrix(1:12, nrow = 3)
> matrix(1:12, ncol = 4)
```

Matrices are loaded by columns, by default.

```
> mat <- matrix(1:12, nrow = 3)
> mat
> mat[1, 1]
> mat[, 1]
> mat[1, ]
> dim(mat)
> a <- 1:5
> b <- 6:10
> cbind(a, b)
> rbind(a, b)
```

**10. Dates.**

The standard form for displaying dates is `yyyy-mm-dd`. R uses the convention that dates are stored as days since `1970-01-01` so dates before this are stored as negative numbers.

```
> mydate <- as.Date("1965-11-15")
> mydate
> as.numeric(mydate)
```

Other formats must be specified.

```
> as.Date("15/11/1965", "%d/%m/%Y")
> as.Date("15nov1965", "%d%b%Y")
```

Help on date formats is obtained with `?strftime`.

```
> library(Epi)
> cal.yr("15nov1965", "%d%b%Y")
> as.Date(cal.yr("15nov1965", "%d%b%Y"))
> format(as.Date("1965-11-15"), "%d%b%Y")
```

The date must be of class `"Date"` for `format()` to work.

### 11. Workspace.

Your workspace is in memory not on disk. You can see which objects are in your workspace with the functions `objects()`, `ls()`.

```
> objects()
> ls()
> rm(x, y, mat)
> ls()
> rm(list = ls())
> ls()
```

The last call makes a list of all objects in the workspace and removes them.

### 12. Help.

```
> ?rm
> help.search("regr")
```

## Exercises

Solutions can be obtained with the instruction

```
> source("solutions/btools.r", echo = TRUE)
```

1. Display the natural log of 10. How about logs to the base 10 (or 2)?
2. Display the value of $\sqrt{3^2 + 4^2}$.
3. Create a vector `v` with components `1, -1, 2, -2`, and display the contents of `v`.
4. Display the second element of `v`. Display the vector containing all but the second element of `v`.
5. Display the vector containing just the positive elements of `v`. Display the indices of these elements.
6. Create the vector `w` with components `(0, 1, 2, 3, 5, 10, 15, ..., 75)`, and find its length.
7. Insert the number `80` at the end of `w`.
8. Insert the number `4` as the 5'th element of `w`.
9. Create the vector `("A", "A", "B", "B", "A")` by indexing `c("A","B")` with a vector.
10. Use `rnorm` to create 100 observations from a normal distribution with mean 0 and variance 1 and assign them to the vector `v`. Verify that the length of `v` is 100, and use the functions `mean()` and `var()` to find the mean and variance of its elements. Verify your answers by calculating the mean and variance directly from the vector `v`, using the function `sum()`.[1]
11. Make a list containing your first and second name as elements and assign it to `name`. Look at the contents of `name` with `str()`.
12. Display your first and second names separately using `[[ ]]`.

[1] The formula is $\sum(v_i - \bar{v})^2/(n-1)$.

13. Re-do this but this time using `"first"` to name your first name and `"second"` to name your second name. Look at the new contents of `name` with `str()`.

14. Display your first and second names separately using the `$` notation.

15. Rename the list `name` as `myname` and remove `name`.

16. Create the vector `("A","B","A","B", ...)` in which the pair `("A","B")` occurs 20 times.

17. Convert `"2000-12-25"` to date format. Check how it is stored.

18. Convert `"25/12/2000"` to date format.

19. Convert `"25dec2000"` to date format.

20. Calculate the number of days which have elapsed since your birthday.

21. Use `cal.yr()` to convert the date `"2008-10-06"` to decimal years.

## 2 Data frames

As an example of a data frame we shall use the `births` data:

- `id`: Identity number for mother and baby.
- `bweight`: Birth weight of baby.
- `lowbw`: Indicator for birth weight less than 2500 g.
- `gestwks`: Gestation period in weeks.
- `preterm`: Indicator for gestation period less than 37 weeks.
- `matage`: Maternal age.
- `hyp`: Indicator for maternal hypertension.
- `sex`: Sex of baby: `1 = Male, 2 = Female`.

The variables in a data frame must all be of the same length, and the values taken by a variable must all be numeric or all be character.

### Summary

- `dir()` Returns the list of files in your working directory.
- `load("data/file")` Loads a data frame previously saved in the subdirectory `"data"` of your working directory.
- `str(x)` Returns the internal structure of a data frame `x`.
- `head(x)` Returns the first 6 rows of `x`.
- `names(x)` Returns the names of the variables in `x`.
- `dim(x)` Returns the number of rows and columns in `x`.
- `nrow(x)` Returns the number of rows in `x`.
- `ncol(x)` Returns the number of columns in `x`.
- `NA` Code for not available (missing).
- `summary(x)` Returns a summary of the variables in the data frame `x`.
- `with(x, expr)` With a data frame `x`, evaluate `expr`. Does not incorporate changes to `x`.
- `within(x, expr)` Within a data frame `x`, evaluate `expr`. Does incorporate changes to `x`.
- `subset(x, ...)` Returns a subset of `x` satisfying a logical condition.
- `is.na()` Checks for missing values.
- `na.omit(x)` Reduces a data frame to complete cases.

## Examples

**1. Data frames.** The data frame `births` has already been saved in the file `births.rda` in a subdirectory `data` of your working directory. The extension `rda` refers to "R data".

```
> getwd()
> dir("data")
> load("data/births.rda")
> head(births)
> names(births)
> dim(births)
> nrow(births)
> ncol(births)
> summary(births)
```

**2. Storage.** A data frame is stored as a list of vectors (variables) which can be referenced by name.

```
> str(births)
> births$hyp
> summary(births$hyp)
```

**3. Avoiding the `$` notation.**

```
> with(births, summary(hyp))
> with(births, summary(cbind(hyp, sex)))
```

The function `with()` temporarily sets up a data frame as the default place to look up variables. Alterations to the temporary data frame are not carried over to the original.

**4. Missing values.** You cannot test directly for missing values in `x`; instead use the function `is.na(x)`.

```
> is.na(births$gestwks)
> with(births, is.na(gestwks))
> subset(births, is.na(gestwks))
> nrow(subset(births, is.na(gestwks)))
```

**5. Referencing parts of a data frame.**

```
> births
> births[1, "bweight"]
> births[1, 2]
> births[1:10, "bweight"]
> births[, "bweight"]
> births[1, -2]
```

**6. Adding new variables or removing old ones.** There are two ways of creating or removing variables.

```
> births$logbw <- log(births$bweight)
> names(births)
> births$logbw <- NULL
> names(births)
```

The variable `logbw` has been removed from the data frame.

```
> births <- within(births, logbw <- log(bweight))
> names(births)
> births <- within(births, rm(logbw))
```

The function `within()` incorporates any changes to the data frame, unlike `with()`.

```
> births <- transform(births, logbw = log(bweight))
```

Note that the transformation must be done using `=` not `<-`. The advantage of `transform` is that several variables can be transformed with a single call to `transform`. It is more difficult to do this using `within`.

### Exercises

1. Display the data on the variable `gestwks` for row 7 in the `births` data frame.
2. Display all the data in row 7.
3. Display the first 10 rows of the data on the variable `gestwks`.
4. Create a logical variable called `early`, in your workspace, according to whether `gestwks` is less than 30 or not. Repeat, but this time make sure the new variable is in the data frame.
5. Display the `id` numbers of women with `gestwks` less than 30 weeks.
6. Create a new data frame containing data from women who were hypertensive.
7. Create a new data frame containing data from women whose gestation time was not missing.
8. Create a new data frame from `births` with no missing values (i.e. complete cases only).
9. Create a new variable called `gender`, in `births`, which takes the value `"M"` when the subject is male and `"F"` when the subject is female. Hint: look at basic tools, Exercise 4.

## 3 Reading data from files

Before reading in the data you should inspect the file in a text editor and ask three questions:

1. Are variable names included in the file?
2. How are the columns in the file separated?
3. How are missing values represented?

There are several things which can go wrong when reading data files. Here are a few common ones:

- A variable whose values are all numeric is read as a numeric variable, but if any of the values include characters the whole variable is read as characters. The only exception is the value `NA`.
- In particular, if an isolated dot is used for "missing" the whole variable will be read as characters, unless you specify `na.strings = "."`.
- If you forget the `header = TRUE` argument all the variables will be read as characters with the variable names `V1, V2`, etc.
- If you forget the `sep` argument the default `sep = ""` is used, i.e. white space which includes spaces and tabs.

### Summary

- `read.table("file")` Reads data from a file as a data frame.
- `write.table(x, "file")` Writes a data frame `x` to a file.
- `save(x, "file")` Saves a data frame `x` to a file.
- `load("file")` Loads the data frame saved in `"file"`.
- `library(foreign)` Loads the library for import/export from other packages.
- `read.dta("file")` Reads a file which has been saved in Stata format.
- `write.dta(x, "file")` Writes a data frame to `"file"` in Stata format.

### Examples

**1. Clear the workspace.**

```
> rm(list = ls())
```

**2. Reading a data frame from a file.** In each of the files `births.csv`, `births.dat`, and `births.tab` the first row contains the names of the variables (`header = TRUE`). The separators are comma, white space, and tab respectively, and an isolated dot (`.`) is used to indicate missing values.

```
> births <- read.table("data/births.csv",
+   header = TRUE, sep = ",", na.strings = ".")
> head(births)
> rm(births)

> births <- read.table("data/births.dat",
+   header = TRUE, sep = "", na.strings = ".")
> head(births)
> rm(births)

> births <- read.table("data/births.tab",
+   header = TRUE, sep = "\t", na.strings = ".")
> head(births)

> save(births, file = "data/births2.rda")
> rm(births)
> load("data/births2.rda")
> head(births)
```

The commands `save()` and `load()` can be used with any R objects.

**3. Importing a data frame from another package.** The file `births.dta` contains the `births` data in Stata format. It can be imported using a function from the `foreign` package.

```
> rm(births)
> library(foreign)
> births <- read.dta("data/births.dta")
> head(births)
```

Note that all decisions about headers, separators, etc., were taken when the data was read into Stata.

### Exercises

1. The file `fem.dat`, in your data directory, contains data on 118 female psychiatric patients. Inspect this file and answer the three basic questions needed when reading data.
2. Read in the data using `read.table()` and assign the result to `fem`.
3. Summarize the data and make sure you have dealt with missing values in `IQ` correctly.
4. Try reading the data forgetting the option `header = TRUE`. What has happened?
5. Still forgetting the option `header = TRUE`, include the option `as.is = TRUE`. What has happened now?
6. The file `fem.dta` contains these data in Stata format. Read the data from this file.
7. The file `fem.tpt` contains these data in SAS XPORT format. Read the data from this file.
8. Have a look at the contents of the file `fem-dot.dat` and read it in, assigning the result to `fem`.

## 4 Factors

It is often important to distinguish between numeric variables which measure something and factors which are used to group subjects into categories. This is because factors are treated differently from numeric variables in many functions. Factors are particularly important in statistical models.

### Summary

- `factor()` Converts a variable to a factor.
- `levels()` Returns the levels of a factor.
- `cut()` Creates a factor from a numeric variable by cutting.

### Examples

**1. Factors, levels and labels.**

```
> load("data/births.rda")
> births$hypf <- factor(births$hyp,
+   labels = c("normal", "hyper"))
> str(births$hypf)
> births$hypf
```

The factor `hypf` has levels labelled as `"normal"` and `"hyper"` but stored as 1/2. When the factor is printed the levels replace the codes.

```
> births$hypf <- factor(births$hyp)
> births$hypf
> str(births$hypf)
```

If no labels are specified the original values of `hyp` are used. This can be confusing when the original values are numeric.

**2. On the fly.**

```
> with(births,
+   summary(factor(hyp,
+   labels = c("normal", "hyper"))))
```

It is always possible to use `factor()` on the fly inside expressions, rather than creating new variables.

**3. Grouping.** Variables taking a range of values can be converted to factors by grouping with the `cut()` function.

```
> births$agegrp <- cut(births$matage,
+   breaks = c(20, 30, 35, 40, 45),
+   right = FALSE)
> str(births$agegrp)
```

When `right = FALSE` the grouping intervals include the right hand end but not the left hand end, and the levels of the factor are labelled `[20,30)`, `[30,35)`, etc.

### Exercises

1. Using the `births` data convert the variable `sex` (coded 1 = male, 2 = female) to a factor called `sexf`. Check the result, and check how the factor is stored. Is `sexf` in your workspace or in the data frame `births`?
2. Do the same but label the levels `"male"` and `"female"`. Print the contents of `sexf`.
3. Print the contents of the factor formed from `sex`, using labels, but do it on the fly, without creating a new object `sexf`.
4. Group the values of the variable `gestwks` into intervals `[20,35)`, `[35,37)`, `[37,39)`, `[39,45)`, and assign the result to a factor called `gest4`. Use the option `right = FALSE`.
5. Group the values of the variable `gestwks` into 5 equal intervals. Try this also with `labels = FALSE`.

## 5 Tables

### Summary

- `table()` Makes tables of frequency distributions.
- `as.data.frame()` Converts a table into a data frame.
- `prop.table()` Replaces frequencies by proportions.
- `library(Epi)` Loads the `Epi` package.
- `stat.table()` General purpose function for producing tables available from the `Epi` package.

## Examples

**1. Tables of frequencies using `table()`.**

```
> load("data/births.rda")
> with(births, table(sex, hyp))
> tb <- with(births, table(sex, hyp))
> prop.table(tb, 1)
> prop.table(tb, 2)
> 100 * prop.table(tb, 1)
> round(100 * prop.table(tb, 1), 2)
> as.data.frame(tb)
> tb <- with(births, table(sex, hyp, lowbw))
> as.data.frame(tb)
```

**2. Tables of means, and other things, using `stat.table`.**

```
> library(Epi)
> stat.table(sex, contents = mean(bweight),
+   data = births)

> stat.table(sex,
+   contents = list(count(), mean(bweight)),
+   data = births)

> stat.table(sex,
+   contents = list(N = count(),
+   ‘Mean birth weight‘ = mean(bweight)),
+   data = births)

> stat.table(list(gender = sex),
+   contents = list(N = count(),
+   ‘Mean birth weight‘ = mean(bweight)),
+   data = births)

> stat.table(sex,
+   contents = list(count(), mean(bweight)),
+   margin = TRUE,
+   data = births)

> stat.table(list(sex, hyp),
+   contents = list(count(), mean(bweight)),
+   margin = c(TRUE, FALSE),
+   data = births)
```

**3. Tables of percents for a binary response coded 0/1.**

```
> stat.table(sex, mean(100 * lowbw),
+   data = births)
```

This only works because `lowbw` is coded 0/1, with 1 for low birth weight.

```
> stat.table(sex, ratio(lowbw, 1 - lowbw),
+   data = births)
```

This produces a table of the odds of low birth weight by sex.

```
> odds.tab <- stat.table(sex,
+   list(odds = ratio(lowbw, 1 - lowbw)),
+   data = births)
> print(odds.tab)
> print(odds.tab, width = 15, digits = 3)
```

The `print` function offers more control over how the table is printed.

### Exercises

1. Start by loading the `births` data. Babies are classified as pre-term if they are born before 37 weeks. Make a table showing the frequencies of normal and pre-term babies using `table()`. Convert the frequencies to proportions.
2. Make a table showing the joint frequency distribution of `preterm` and `hyper`. Convert this to a data frame.
3. Make a table of mean birth weight by `preterm`. Improve the labelling of the columns.
4. Using the fact that `preterm` is coded 0/1, make a table of the proportion of pre-term babies by `hyp` and `sex`. Choose which margin to show.
5. Group the values of `matage` in a factor, and prepare a table of proportions of pre-term babies by the levels of this factor. Show the margin.
6. Repeat the previous table using odds in place of proportions.
7. Explore some of the other things `stat.table` can put in a table.

## 6 Graphics in R

The philosophy of graphs in R is different from other packages. There are two kinds of plotting functions in R:

- High level functions that generate a new plot, e.g. `hist()` and `plot()`.
- Low level functions that add extra things to an existing plot, e.g. `lines()` and `text()`.

The normal procedure for making a graph in R is to make a fairly simple initial plot and then add things to it. Once added they cannot be removed so it is best to use a script which can be edited and run again from the beginning.

- The results from graphics functions cannot be placed in objects.
- Instead they go to a graphics device, which may be a window or a file.
- Several graphics devices can be open at once.
- `dev.off()` turns off the current device – important when this is a file.
- R allows you to fine-tune how a graph looks with options to control colors, plotting character sizes, etc.
- Most of the options can be used in both high and low level functions, but only hold while that function is executed.
- The `par()` function lists the defaults for all of the graphical parameters. This function can be used to change the defaults but the change now holds for the rest of the session.

### Classes and generic functions

All objects in R are arranged in classes. When a function acts on an object it first checks the class of the object to see whether the operation is allowed for that class of objects.

Generic functions deal with many different classes of objects. For each class the generic function selects a method which is specially suited to that class.

- `plot` is an example of a generic function.
- `plot(x)` first checks the class of `x` and then selects the appropriate plot method, for example `plot.ecdf` if `x` has class `"ecdf"`.
- `methods(plot)` shows all the plot methods available for the different classes of object.

The only time the user needs to know about this is when using help. Calling `help(plot)` gives general help on features common to all the plot methods but for specific features it is necessary to call help on the specific plot method, e.g. by using `help(plot.ecdf)`.

## 7 High level graphics functions

### Summary

#### Attaching a data frame and sourcing a file

- `attach(x)` Attaches the data frame `x` in position 2 of the search path.
- `detach(x)` Detaches the data frame `x` from the search path.
- `search()` Returns the search path.
- `source("file", echo = TRUE)` Executes the commands in a file.

#### Graphics functions

- `hist(x)` Returns a histogram of the values of a vector `x`.
- `boxplot(x)` Returns a boxplot of the values of a vector `x`.
- `plot(x, y)` Plots `y` vs `x`.
- `plot(y ~ x)` Plots `y` vs `x`.
- `ecdf(x)` Returns the empirical cumulative distribution function of `x`.
- `plot(ecdf(x))` Plots the ecdf of `x`.

#### Graphics parameters (`tag = value`)

- `main` Main title.
- `sub` Sub title.
- `pch` Plotting character to be used.
- `xlab`, `ylab` Labels for axes.
- `xlim`, `ylim` Limits for axes.
- `type` `"l"` for lines, `"p"` for points, `"b"` for both, `"n"` for none, ...

- `cex` Controls size of plotting character relative to 1.
- `las` Controls orientation of axis labels.
- `col` Controls color of plotting character.
- `lty` Controls type of line.
- `lwd` Controls line width.
- `add = TRUE` Adds the next graph to the current plot.
- `log = "y"` y-axis on logarithmic scale.
- `mar` Controls margins using numbers of lines. Default = `c(5, 4, 4, 2) + 0.1`.
- `mfrow`, `mfcol` Vector of form `(nr, nc)` which controls numbers and positions of graphs.
- `par()` Used for querying and changing the default graphics parameters.

### Saving graphs

- `X11()` Opens a graph window (unix or windows).
- `pdf("fname")` Opens a pdf file in your working directory.
- `dev.off()` Turns off the current device.

## Examples

**1. Attaching a data frame.** A data frame can be attached in position 2 of the search path. This makes a copy of the variables in position 2 so they can be referenced without specifying the data frame.

```
> attach(births)
> search()
> objects(2)
> detach(births)
```

**2. Creating new variables with an attached data frame.**

```
> attach(births)
> logbw <- log(bweight)
> objects(1)
> head(births)
> objects(2)
```

`logbw` is in your workspace but not in the data frame and not in position 2.

```
> births$logbw <- log(bweight)
> head(births)
> objects(2)
> detach(births)
```

`logbw` is now in `births` but still not in position 2. To get it in position 2 you would have to detach `births` and then attach it again.

**3. High level graphs.** Making graphs is one of the few occasions where it is worth attaching a data frame, but do not forget to detach it at the end, and never include `attach()` calls in a script.

Note that all calls to a high level graph function open a new window.

```
> load("data/births.rda")
> attach(births)
> hist(gestwks)
> hist(bweight, br = c(0, 1000, 2000, 3000, 4000, 5000))
> plot(gestwks, bweight)
> plot(ecdf(gestwks))
```

**4. Using a formula.**

```
> plot(bweight ~ gestwks)
> plot(bweight ~ gestwks, data = births)
> boxplot(bweight ~ hyp, data = births)
```

The expression `bweight ~ gestwks` is a simple example of a model formula (birth weight depends on gestation). The plot functions in this form take a `data` argument, so there is no advantage to attaching the data frame. Unfortunately not all graphics functions allow a formula.

**5. Altering graphics parameters within high level functions.**

```
> colors()
> plot(1:25, pch = 1:25)
> hist(bweight, col = "gray", border = "white")
> plot(bweight ~ gestwks, pch = 4)
> plot(bweight ~ gestwks, pch = 4, col = "green")
> plot(bweight ~ gestwks,
+   main = "Birth weight vs Gestation",
+   xlab = "Gestation in weeks",
+   ylab = "Birth weight in g")
> plot(ecdf(gestwks), cex = 0.5)
```

`cex` refers to the relative size of the plotting character.

**6. Altering default values with `par()`.**

```
> par(pch = 2, col = "green")
> plot(bweight ~ gestwks)
> par(pch = 1, col = "black")
```

The original values of the parameters must be restored unless you want to carry on using the new ones.

**7. Line plots.**

```
> x <- 1:5
> y <- c(2.3, 4.5, 5.5, 6.2, 6.3)
> plot(x, y)
> plot(x, y, type = "l")
> plot(x, y, type = "b")
> plot(x, y, type = "h")
> plot(x, y, type = "n")
```

Starting a plot with nothing but the axes allows you to customize the rest.

**8. Saving graphs.**

```
> pdf("test.pdf")
> plot(bweight ~ gestwks)
> dev.off()
```

The graph will be saved in your working directory.

**9. Detaching.**

```
> detach(births)
> search()
```

Do not forget to detach when it is all over!

### Exercises

1. Load and attach the `births` data.
2. Make a box plot of the maternal age of mothers. Change the color of the box.
3. Make a box plot of the maternal age of mothers by `hyp`. Change the color of the boxes to blue for `normal` and red for `hyper`.
4. Make a histogram of maternal age, in which the x-axis runs from 20 to 45, and the breaks are at 5 year intervals from 20. Color the rectangles grey.
5. Plot the ecdf of maternal age.
6. Make two plots on the same graph showing the ecdf of maternal age for each level of `hyp`.
7. Detach the `births` data.

## 8 Low level graphics functions

### Summary

- `text()` Adds text.
- `mtext()` Adds text to the margins.
- `legend()` Adds a legend.
- `points()` Adds points.
- `lines()` Adds lines.
- `arrows()` Adds arrows.
- `segments()` Adds segments.
- `abline()` Adds a line to a plot.

## Examples

### 1. Adding text.

```
> attach(births)
> plot(bweight ~ gestwks, pch = 4, col = "green")
> text(30, 4000, "Nice green points")

> plot(bweight[1:10] ~ gestwks[1:10],
+   pch = 4, col = "green")
> text(gestwks[1:10], bweight[1:10],
+   pos = "4", labels = 1:10)

> plot(bweight[1:10] ~ gestwks[1:10],
+   pch = 4, col = "green",
+   xlim = c(35, 50))
> text(gestwks[1:10], bweight[1:10],
+   pos = "4", labels = bweight[1:10])
```

### 2. Empty graph, then points and legend.

```
> plot(gestwks, bweight, type = "n")
> points(gestwks[sex == 1], bweight[sex == 1],
+   col = "red")
> points(gestwks[sex == 2], bweight[sex == 2],
+   col = "blue")
> legend(25, 3000, legend = c("Boys", "Girls"),
+   pch = 1, col = c("red", "blue"))
```

Note that the default plotting character must be specified in the legend.

### 3. Adding lines.

```
> abline(v = 30)
> abline(h = 2000)
> arrows(35, 1000, 40, 1500, col = "magenta")
> segments(35, 1000, 40, 1200, col = "darkgreen")
```

### 4. Adding a line of best fit.

```
> plot(bweight ~ gestwks)
> abline(lm(bweight ~ gestwks))
```

The function `lm()` returns the line of best fit – see later.

### 5. Do not forget to detach `births`.

```
> detach(births)
```

## Exercises

1. Load the `births` data and attach it.
2. Make a box plot of the maternal age of mothers by `hyp` with blue boxes for `normal` and red for `hyper`. Add a legend to this plot.

3. Make a plot of birth weight vs maternal age, and add a horizontal line through the mean birth weight.

4. Make an empty plot of `bweight` vs `gestwks`, and set the vertical axis to run from 0 to 5000.

5. Add points to this plot but color them blue when `bweight` is greater than 2000, and red otherwise. Add a legend to the plot.

6. Detach the `births` data.

# 9 Effects

Many statistical problems can be expressed in terms of the effect of various explanatory variables on a response variable. The response variable must be numeric. Main types are

- Metric (a measurement with units).
- Binary (two values coded 0/1).
- Failure (does the subject fail at end of follow-up).
- Count (aggregated failure data).

Explanatory variables can be

- Numeric.
- Factor.

A further important aspect of explanatory variables is the role they will play in the analysis.

- Primary role: exposure.
- Secondary role: confounder.

The word *effect* is a general term referring to ways of comparing the values of the response variable at different levels of an explanatory variable. The main measures of effect are:

- Differences in means for a metric response.
- Ratios of odds for a binary response.
- Ratios of rates for a failure or count response.

The function `effx` from the `Epi` package is a useful way of introducing the main concepts.

## Examples

### 1. Housekeeping.

```
> library(Epi)
> load("data/births.rda")

> births <- within(births,
+   hyp <- factor(hyp,
+   labels = c("normal", "hyper")))
```

```
> births <- within(births,
+   sex <- factor(sex,
+   labels = c("M", "F")))

> births <- within(births,
+   agegrp <- cut(matage,
+   breaks = c(20, 25, 30, 35, 40, 45),
+   right = FALSE))

> births <- within(births,
+   gest4 <- cut(gestwks,
+   breaks = c(20, 35, 37, 39, 45),
+   right = FALSE))
```

Categorical variables should be converted to factors.

**2. Effects of sex on birth weight.**

```
> effx(response = bweight, typ = "metric",
+   exposure = sex, data = births)
```

The effect of `sex` on `bweight`, measured as a difference in means, is about −197. The p-value refers to the test that there is no effect of sex on birth weight.

```
> stat.table(sex, mean(bweight), data = births)
```

This verifies the result from `effx`.

**3. Effects of sex on `lowbw`.**

```
> effx(response = lowbw, typ = "binary",
+   exposure = sex, data = births)
```

The effect of `sex` on `lowbw`, measured as an odds ratio, is about 1.43.

```
> stat.table(sex,
+   list(odds = ratio(lowbw, 1 - lowbw, 100)),
+   data = births)
```

**4. Factors on more than two levels.**

```
> effx(response = bweight, typ = "metric",
+   exposure = gest4, data = births)
```

There are now 3 effects (for levels 2, 3, 4 vs 1).

```
> stat.table(gest4, mean(bweight), data = births)
```

**5. Effects of `hyp` on `bweight` stratified by `sex`.**

```
> effx(bweight, type = "metric",
+   exposure = hyp, strata = sex,
+   data = births)
```

The effects of `hyp` in the different strata defined by `sex` can be compared. The test for effect modification is a test of whether they differ significantly.

**6. Controlling the effect of `hyp` for `sex`.**

```
> effx(bweight, type = "metric",
+   exposure = hyp, control = sex,
+   data = births)
```

The effect of `hyp` on `bweight` controlled for `sex` is obtained by combining stratum-specific effects when they are similar. There can be more than one control variable, e.g. `control = list(sex, agegrp)`.

**7. Numeric exposures.**

```
> effx(response = bweight, type = "metric",
+   exposure = gestwks, data = births)

> effx(response = lowbw, type = "binary",
+   exposure = gestwks, data = births)
```

The linear effect of `gestwks` on `lowbw` is a reduction in odds by a factor per extra week of gestation.

**8. Stratifying a linear effect by a factor.**

```
> effx(lowbw, type = "binary",
+   exposure = gestwks, strata = agegrp,
+   data = births)
```

You can control for a numeric variable by putting it in the `control` list.

### Exercises

1. Load the `births` data and use `effx` to find the effect of `hyp` on `bweight`, and verify your answer using `stat.table`.
2. Use `effx` to find the effect of `hyp` on `lowbw`, and verify your answer using `stat.table`.
3. Find the effects of `gest4` on `lowbw`. Use the option `base = 4` to change the baseline for `gest4` from 1 to 4.
4. Use `effx` to stratify the effect of `hyp` on `lowbw` first by `sex` and then by `gest4`.
5. Control the effect of `hyp` on `lowbw` for `sex`.

## 10 Linear models

The `lm` and `glm` functions in R provide a more versatile approach to estimating effects than `effx` but the output is more difficult to interpret. Important concepts in linear models include model formulae, families, and link functions.

- `lm(formula)` for linear regression.
- A typical formula is `y ~ A + B` which means that the response depends additively on the two variables `A` and `B`.
- `glm(formula, family = binomial)` for logistic regression.
- The default link function for logistic is log odds. Coefficients are log odds ratios.
- `glm(formula, family = poisson)` for Poisson regression.
- The default link function for Poisson is log. Coefficients are log rate ratios.

## Summary

- `y ~ x` A model formula: `y` depends on `x`.
- `lm()` Fits a linear model.
- `glm()` Fits a generalized linear model.
- `coef()` Returns the coefficients from a linear model.
- `anova()` Compares the fit of two nested models.
- `relevel()` Changes the baseline for factors.

## Examples

### 1. Housekeeping.

```
> load("data/births.rda")

> births <- within(births,
+   hyp <- factor(hyp,
+   labels = c("normal", "hyper")))

> births <- within(births,
+   sex <- factor(sex,
+   labels = c("M", "F")))

> births <- within(births,
+   gest4 <- cut(gestwks,
+   breaks = c(20, 35, 37, 39, 45),
+   right = FALSE))
```

Categorical variables should be converted to factors before being used in linear models.

### 2. Linear effects.

```
> m <- lm(bweight ~ gestwks, data = births)
> summary(m)
> coef(m)
```

One extra week of gestation produces an extra amount of baby weight (about 197 g).

### 3. Explanatory variable is a factor.

```
> m <- lm(bweight ~ hyp, data = births)
> summary(m)
> coef(m)
```

The name of the coefficient is made up from the name of the variable plus the second level of the factor. The first level of a factor is used as the baseline, by default.

### 4. Omitting the intercept.

```
> m <- lm(bweight ~ -1 + hyp, data = births)
> coef(m)
```

This gives the mean `bweight` at the two levels of `hyp`.

**5. Generalized linear models.**

```
> m <- glm(lowbw ~ hyp,
+   family = binomial, data = births)
> exp(coef(m))
```

Apart from the intercept the coefficients are log odds ratios so must be exponentiated to get odds ratios.

**6. Using `ci.lin` from the `Epi` package.**

```
> library(Epi)
> ci.lin(m, Exp = TRUE)
> ci.lin(m, Exp = TRUE)[-1, 5:7]
```

Using `ci.lin` it is easy to trim the output down to what is needed.

**7. Controlling.**

```
> m <- glm(lowbw ~ hyp + sex,
+   fam = binomial, data = births)
> exp(coef(m))
```

Controlling the effect of `hyp` on `lowbw` for `sex` assumes no interaction.

**8. Interaction (effect modification).**

```
> m <- glm(lowbw ~ hyp * sex,
+   fam = binomial, data = births)
> exp(coef(m))
```

To include interaction between `A` and `B` in a model formula use `A * B`.

**9. Testing for interaction.**

```
> m1 <- glm(lowbw ~ hyp + sex,
+   fam = binomial, data = births)
> m2 <- glm(lowbw ~ hyp * sex,
+   fam = binomial, data = births)
> anova(m1, m2, test = "Chisq")
```

**10. Stratified effects.**

```
> m <- glm(lowbw ~ sex/hyp,
+   fam = binomial, data = births)
> exp(coef(m))
```

When there is strong interaction it may be best to report stratified effects, but note that the variable defining the strata must be a factor.

### Exercises

1. Load the `births` data and cut the variable `gestwks` into factor `gest4` using break points `(20, 35, 37, 39, 45)`.

2. With the `births` data use `stat.table` to prepare a table of the odds of `lowbw` by `gest4`.

3. Find the effects of `gest4` on `lowbw` as odds ratios. What baseline is being used for these odds ratios?

4. Use the function `relevel()` to change the baseline to the highest level of gestation time. You can do this on the fly by including `relevel(gest4, ref = 4)` in the model formula instead of `gest4`.

5. Find the effect of `hyp` on `lowbw` as an odds ratio.

6. Control the effect of `hyp` on `lowbw` for `gestwks`.

7. Control the effect of `hyp` on `lowbw` for `gest4`.

8. The data in the file `"data/ucb.rda"` refers to aggregate (frequency) data on applications to graduate school in Berkeley classified by admission, department, and gender. The variable `reject` is coded 0 for admitted and 1 for rejected. Load these data and find the odds ratio for the effect of gender on rejection (you will need to use weights equal to the variable `freq`). What are your conclusions? Now stratify this effect by department. What are your conclusions now? Control the effect of gender on rejection for `dept` and report your final conclusions.

## 11 Survival curves

### Summary

- `library(survival)` Loads the `survival` package.
- `Surv()` Creates a survival time object.
- `survfit()` Calculates survival curves.
- `coxph()` Fits a proportional hazards model.

### Examples

**1. The liver data.** In the data set `liver`, 184 subjects were enrolled in a randomized clinical trial for the treatment of primary biliary cirrhosis. Variables included

- `id` Identity number.
- `y` Follow up time in years.
- `d` Status at end of follow up, 1 = dead, 0 = alive.
- `treat` Treatment, 1 = placebo, 2 = active.
- `age` Age at start of study in years.
- `cirrhos` Cirrhosis, 1 = present, 0 = absent.
- `bilirub` Bilirubin, 1 = present, 0 = absent.

Subjects were randomly allocated to two treatments, active and placebo, identified by the variable `treat`. The variable `y` contains the length of follow-up and `d` is an indicator for an event.

```
> load("data/liver.rda")
> names(liver)
> head(liver)
> str(liver)
```

Note that `treat` and `cirrhos` are factors.

**2. Overall mortality rates.**

```
> with(liver, sum(d))
> with(liver, sum(y))
> with(liver, sum(d) / sum(y))

> stat.table(treat,
+   contents = list(rate = ratio(d, y)),
+   data = liver)

> stat.table(treat,
+   contents = list(rate = ratio(d, y, 1000)),
+   data = liver)
```

**3. The response.**

- The response in survival analysis is made up of the follow-up time and the event indicator.
- The event indicator should be coded 0 for no event (censored) and 1 for an event.
- The response is created with `Surv(time, event)` from the `survival` package.
- A second time variable can be used to allow for late entry.

The response in survival time data usually consists of two pieces of information – the time which the subject spends in the study and the subject's status at the end of this time.

```
> library(survival)
> st <- with(liver, Surv(y, d))
> st
```

In this case the time variable (in years) is `y` and the failure variable is `d`, coded 1 for death from any cause, 0 otherwise. The survival time object `st` combines the two aspects of the response. The output from `st` consists of the survival times with a "+" where these have been censored.

**4. Kaplan–Meier survival function.**

```
> fit <- survfit(st ~ 1)
> summary(fit)
> plot(fit)

> fit <- survfit(st ~ treat, data = liver)
> fit
> plot(fit, col = c("red", "blue"))
> legend(2, 0.2,
+   legend = c("placebo", "active"),
+   text.col = c("red", "blue"))
```

The left hand side of the model formula must be a survival time object.

**5. Plots of cumulative hazard.**

```
> plot(fit, col = c("red", "blue"),
+   fun = "cumhaz")
> legend(2, 1,
+   legend = c("placebo", "active"),
+   text.col = c("red", "blue"))
```

The gradient of the cumulative hazard curve is the rate. In some ways plots of cumulative hazard are preferable to Kaplan–Meier plots.

**6. Cox proportional hazards model.**

```
> coxph(st ~ treat, data = liver)
```

This can be used to estimate the effect of treatment as a rate ratio, assuming proportional hazards.

**7. Checking proportional hazards.**

```
> fit <- survfit(st ~ treat, data = liver)
> plot(fit, fun = "cumhaz", log = "y")
```

For proportional hazards the curves should be parallel.

**8. Test for proportional hazards.**

```
> m <- coxph(st ~ treat, data = liver)
> cox.zph(m)
```

## Exercises

1. Use the `liver` data to plot survival curves according to whether the subject had cirrhosis at the start of the trial or not.
2. Plot the cumulative hazard curves by `cirrhos` as well.
3. Use the plot of cumulative hazards with a log scale to check proportional hazards.
4. Make a test of proportional hazards.
5. Assuming proportional hazards, find the effect of cirrhosis as a rate ratio.
6. Find the effect of `treat` controlled for `cirrhos`.
7. Assuming linearity, find the effect of age as a rate ratio per year of age.
8. Find the effect of age as a rate ratio per 10 years of age. Hint: include `I(age/10)` in the model formula instead of `age`. The `I()` is necessary to stop R from trying to interpret `age/10` as a model formula.
9. Find the effect of `treat` controlled for age.

# 12 Writing simple functions

## Summary

- `function(x) { ... }` Definition of a function.

## Examples

**1. A simple function to calculate** $\log\big(p/(1-p)\big)$**.**

```
> logit <- function(p) log(p/(1 - p))
> logit(0.2)
```

The next step might be to add a check that `p` lies between 0 and 1, and to stop with an error message if not.

```
> logit <- function(p) {
+   if (p < 0 | p > 1) stop("Out of range")
+   return(log(p/(1 - p)))
+ }
> logit(0.2)
> logit(-0.1)
> logit(1.1)
```

We shall add an option `half` such that, when `half` is `TRUE`, the function returns $0.5\log(p/(1-p))$.

```
> logit <- function(p, half = FALSE) {
+   if (p < 0 | p > 1) stop("Out of range")
+   if (half == TRUE)
+     return(0.5 * log(p/(1 - p)))
+   else
+     return(log(p/(1 - p)))
+ }
> logit(0.2)
> logit(0.2, half = TRUE)
```

The argument `half` takes the value `FALSE` by default; when there is no default it means the argument must be given a value in the function call.

## Exercises

1. Write a function to calculate the area of a circle from its radius. Think of a name for this function and check that a function of this name does not already exist.

2. Re-write the function with an argument `radius` which, when true, calculates the area from the radius, but otherwise calculates it from the diameter.

3. Write a function which returns the first element of a vector `x`. Call it `first`.

4. Write a function which returns the last element of a vector `x`. Call it `last`.

5. Write a function which returns the change between the first and last element of a vector `x`. Call it `change`.

# 13 More about data manipulation

## Summary

- `order(x)` Returns the rank order of the elements of a vector `x`.
- `merge()` Merges two data frames.
- `sapply(list, fun)` Applies a function to the elements of a list.
- `tapply(x, index, fun)` Applies a function to groups of the vector `x` defined by `index`.
- `melt()` Defines a data frame in terms of id variables and analysis variables.
- `cast()` Carries out the analysis.

## Examples

### 1. Sorting a data frame by one of its variables.

```
> load("data/births.rda")
> o <- with(births, order(bweight))
> head(o)
> births <- births[o, ]
> head(births)
```

### 2. Merging files.

```
> b1 <- births[, c(1, 2)]
> head(b1)
> b2 <- births[, c(1, 4)]
> head(b2)
```

This creates two data frames from `births`, both containing `id`.

```
> b3 <- merge(b1, b2)
> head(b3)
```

The merge is automatically matched on variables common to both files, but you can use the `by` option to be more specific.

### 3. `sapply`.

```
> sapply(births, class)
> sapply(births, median)
> sapply(births, median, na.rm = TRUE)
```

### 4. `tapply`.

```
> with(births, tapply(bweight, hyp, mean))
> with(births, tapply(bweight, hyp, median))
> with(births, tapply(bweight, hyp, range))
```

**5. `melt` and `cast`.**

```
> library(reshape)
> mbirths <- melt(births,
+   id.var = c("id", "hyp", "sex"),
+   measure.var = c("bweight", "gestwks", "matage"))

> cast(mbirths, hyp + sex ~ variable, mean)
> cast(mbirths, hyp + sex ~ variable,
+   mean, na.rm = TRUE)
```

Variables on the left of `~` define the rows of the result while variables on the right define the columns.

```
> cast(mbirths, variable ~ hyp + sex,
+   mean, na.rm = TRUE)
```

The column names are made up from the values taken by `hyp` and `sex`.

### Exercises

1. Write a function of a vector `x` which returns the number of non-missing values, the mean of the non-missing values, and the standard deviation of the non-missing values. Use `sapply` to apply this function to the variables in the `births` data frame, and put the result in `z`. Use the function `t()` to transpose `z` and then print the result.

2. Melt the `births` data frame with `sex` and `hyp` as the id vars and `bweight`, `gestwks` and `matage` as the measurement vars. Cast it to produce a data frame with median `bweight`, median `gestwks`, and median `matage`, indexed by `hyp`.

3. Load and inspect the data in the file `"data/fevlong.rda"`. These data refer to measurements of `fev` as a percentage of normal for each subject at 3 monthly intervals. Subjects are classified by treatment group. Sort the data by `grp` and `month`.

4. Melt the data with `id`, `month`, and `grp` as id vars and `fev` as measurement var. Cast the data to show the mean `fev` by `grp` and `month`. Use this to plot the mean `fev` vs `month` separately for each group.

5. Cast the data to show the first `fev` reading for each subject by `grp`. Use a function `first` to do this. Make a boxplot of these first values by `grp`.

6. Cast the data to show the last `fev` reading for each subject by `grp`. Use a function `last` to do this. Make a boxplot of these last values by `grp`.

## 14 Packages

### Summary

- `install.packages("pname")` Installs the package `"pname"` from CRAN.
- `library(pname)` Loads the package `pname` from disk.
- `library(help = pname)` Lists the contents of `pname`.

Some useful packages are:

- `Epi` Functions useful in Epidemiology.
- `survival` Functions for survival analysis.
- `foreign` Functions for import and export.
- `xtables` Functions to convert output to LaTeX format.
- `reshape` Functions for reshaping data.

Some are included in the basic installation, e.g. `survival` and `foreign`, so only have to be loaded.

# 15 References

Some useful references are:


1. `https://cran.r-project.org` (CRAN).
2. *R for Beginners*, Emmanuel Paradis (`Paradis-rdebuts_en.pdf` from CRAN).
3. *Data Manipulation with R*, Phil Spector, Springer 2008.
4. *An Introduction to R* (`R-intro.pdf` from CRAN).
5. *R Graphics*, Paul Murrell, Chapman & Hall (expensive).